🧠 一、底层引擎：大语言模型(LM)

核心定义与架构

- **LM全称**：Large Language Model（大语言模型），简称**大模型**
- **底层架构**：基于**Transformer**（2017年Google团队在论文《Attention is All You Need》中提出）
- **工作原理**：本质是**文字接龙游戏**，通过预测下一个概率最高的词生成连续文本

发展里程碑

时间	事件	意义
2017年	Transformer架构提出	奠定大模型技术基础
2022年底	GPT-3.5发布	首个达到可用级别的大模型
2023年3月	GPT-4发布	大幅提升AI能力天花板
2023年后	Claude、Gemini等模型涌现	AI赛道从OpenAI独角戏变为多强竞争

🔤 二、数据处理单元：Token

核心特性

- **定义**：大模型处理文本的**最小单位**，通过Tokenizer（分词器）将文本切分为片段
- **编码过程**：分两步——①文本切分为Token ②映射为Token ID（数字）
- **解码过程**：将Token ID还原为文本（无需切分步骤）

Token与自然语言单位的关系

语言单位	与Token的关系	示例
中文词语	非一一对应，可能被拆分	“工作坊”→“工作”+“坊”
英文单词	常见词通常对应1个Token	“hello”→1个Token
复杂单词	可能被拆分	“helpful”→“help”+“ful”
特殊字符	可能需多个Token表示	“✅”→3个Token

量化参考

- 1个Token ≈ 0.75个英文单词
- 1个Token ≈ 1.5-2个汉字
- 40万Token ≈ 60-80万汉字 或 30万英文单词

🧠 三、临时记忆体：Context

核心概念

- **定义**：大模型每次处理任务时接收的**信息总和**，相当于模型的"临时记忆"
- **组成部分**：用户问题、对话历史、当前输出Token、工具列表、System Prompt等
- **容量限制**：由**Context Window**（上下文窗口）定义，即最大可处理的Token数量

主流模型Context Window对比

模型	Context Window（Token）	约合汉字数量
GPT-5.4	105万	约157.5万
Gemini 3.1 Pro	100万	约150万
Claude Opus 4.6	100万	约150万

突破Context Window限制的方案

- **RAG技术**（检索增强生成）：从知识库中抽取与问题最相关的片段，仅将关键信息送入模型，降低Token消耗

📝 四、指令交互：Prompt

定义与分类

- **Prompt**：给大模型的**问题或指令**，决定模型输出质量
- **Prompt分类**：
  - **User Prompt**：用户输入的具体任务（如"帮我写一首诗"）
  - **System Prompt**：开发者后台配置的**人设与做事规则**（如"你是一个耐心的数学老师，当学生问你数学问题时，不要直接给答案，而是要一步步引导学生思考，帮助他们理解解题的思路"）

Prompt Engineering提示词工程

- **核心原则**：清晰、具体、明确
- **现状**：曾经重要，但现在重要性下降，原因①门槛低（本质是"把话说清楚"）②大模型能力提升，可推测模糊意图

🔧 五、外部能力扩展：Tool

核心作用

- **定义**：大模型调用的**外部函数**，使其能够感知和影响外部环境
- **解决痛点**：弥补大模型无法获取实时信息（如天气）、计算能力有限等弱点

工作流程

1. 用户提问→平台转发（含工具列表）
2. 大模型分析→生成工具调用指令
3. 平台执行调用→获取结果
4. 大模型整理结果→自然语言输出

角色分工

角色	职责
大模型	选择工具、生成参数、归纳结果
工具	执行具体功能（如查询天气）
平台	转发信息、执行工具调用

🔄 六、工具标准化：MCP

- **全称**：Model Context Protocol（模型上下文协议）
- **本质**：理解为**统一的工具接入标准**，解决不同平台工具接入规范不统一的问题。

  比如：openAI、Anthropic、Google三个平台各自有接入规范，用一个要写一个接入规范。所以想搞一个统一的技术规范标准。
  就像手机都统一用type c充电口统一标准
- **价值**：工具开发者只需按MCP规范开发一次，即可在所有支持MCP的平台使用（类比手机Type-C接口）

🤖 七、自主决策系统：Agent

- **Agent定义**：能够**自主规划、自主调用工具**，持续工作直至完成用户任务的系统
- **核心能力**：多步骤推理、工具选择、流程控制
- **代表产品**：Claude Code 、Codex、Gemini CLI等
- **典型构建模式**：React、Plan and Execute等

📋 八、任务定制：Agent Skill

核心功能

- **定义**：给Agent的**说明文档**，包含任务规则、执行步骤、输出格式等
- **结构**：
  - **元数据层**：名称（name）、描述（description）
  - **指令层**：目标、执行步骤、判断规则、输出格式、示例

技术实现

- 存储形式：Markdown文档（文件名必须为"SKILL.md"）
- 存放位置：特定目录（如Claude Code找到用户目录的".claude/skills"文件夹）
- 加载机制：仅在用户问题与技能名称/描述相关时加载完整指令

🔗九、 概念体系关联

![概念体系层级关系]
LM（核心引擎）→ Token（数据单位）→ Context（记忆空间）→ Prompt（交互接口）→ Tool（外部能力）→ MCP（工具标准）→ Agent（决策系统）→ Agent Skill（任务定制）

📝 补充细节

- **Transformer架构**：虽由Google提出，但由OpenAI通过GPT系列引爆应用
- **Token切分原理**：基于BPE（字节对编码）算法，模型自主学习的文本切分规则
- **Agent Skill高级特性**：支持运行代码、引用资源，采用渐进式披露机制节省Token
- **RAG技术**：通过检索相关片段而非全文，有效解决Context Window限制问题